

Table of Contents		
No.	Content	Page
1	Overview	2
2	Data Structures Used	2
3	Module Integration	3
4	Algorithm Complexity	5
5	Sample Output	6
6	Reflection	11
7	Limitations & Assumption	11
8	References	12

Overview

This report outlines the design and implementation of the backend of ZipRide Dispatch System. This system makes use of 4 integrated modules as follows:

1. Graph-based Route Planning - builds a city road network, implements BFS, DFS and finds the shortest possible distance using Dijkstra's Algorithm.
2. Hash-based Passenger and Driver Lookup - stores passenger and driver details and allows fast data retrieval using chaining-based hash tables.
3. Heap-based Pickup Scheduling - Prioritizes pickup requests using Max heap based on the passenger's membership tier and the shortest distance from the driver's location.
4. Sorting Pickup Records - sorts the pickup requests using merge sort and quick sort for end-of-day reporting.

The data flows from route planning to data retrieval to pickup request and finally sorting.

Data Structures Used

Module 1 implements Graph using linked lists. This requires less space and is efficient for sparse graphs. It allows dynamic insertion of nodes and edges compared to adjacency matrix. It also simplifies the traversal for BFS and DFS. It also implements dijkstra algorithms to calculate the shortest time to location.

Module 2 implements a hash table that uses chaining, hence each node is a Linked List. I used chaining as we have already implemented a linked list in the previous module and using chaining to handle collisions is much easier as we don't have to probe and probe to find an empty spot, we can just hash the key and insert it in a linked list. I used 53 as fixed hash array

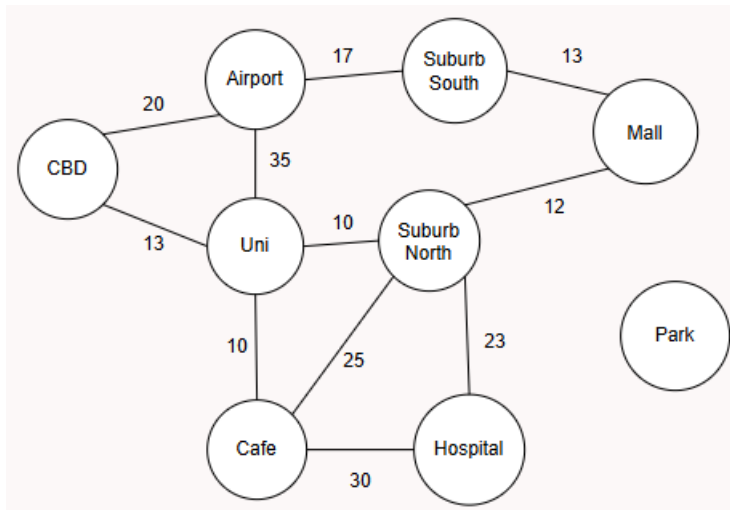
size as prime numbers give a good modulo value and distributes the keys evenly. I didn't implement resizing here.

Module 3 implements a max heap as it makes it easier to access and extract the highest priority request. The priority is calculated using the formula given where we retrieve the passenger's membership tier and the available drivers from the hash table. For each available driver I find the shortest time taken to each passenger's location and then choose the shortest one among all. That chosen driver is then removed from the available driver list, as that driver is now busy.

Module 4 implements the advanced sorting methods, Merge Sort and quick Sort. Merge Sort has a stable time complexity and hence is suitable. For the quick sort, I have used the Median 3 pivot selection strategy as it shows the fastest sorting compared to the other quick sort implementation. When run, the quick sort has a faster performance for random data and ensures in-place sorting, hence doesn't require much storage.

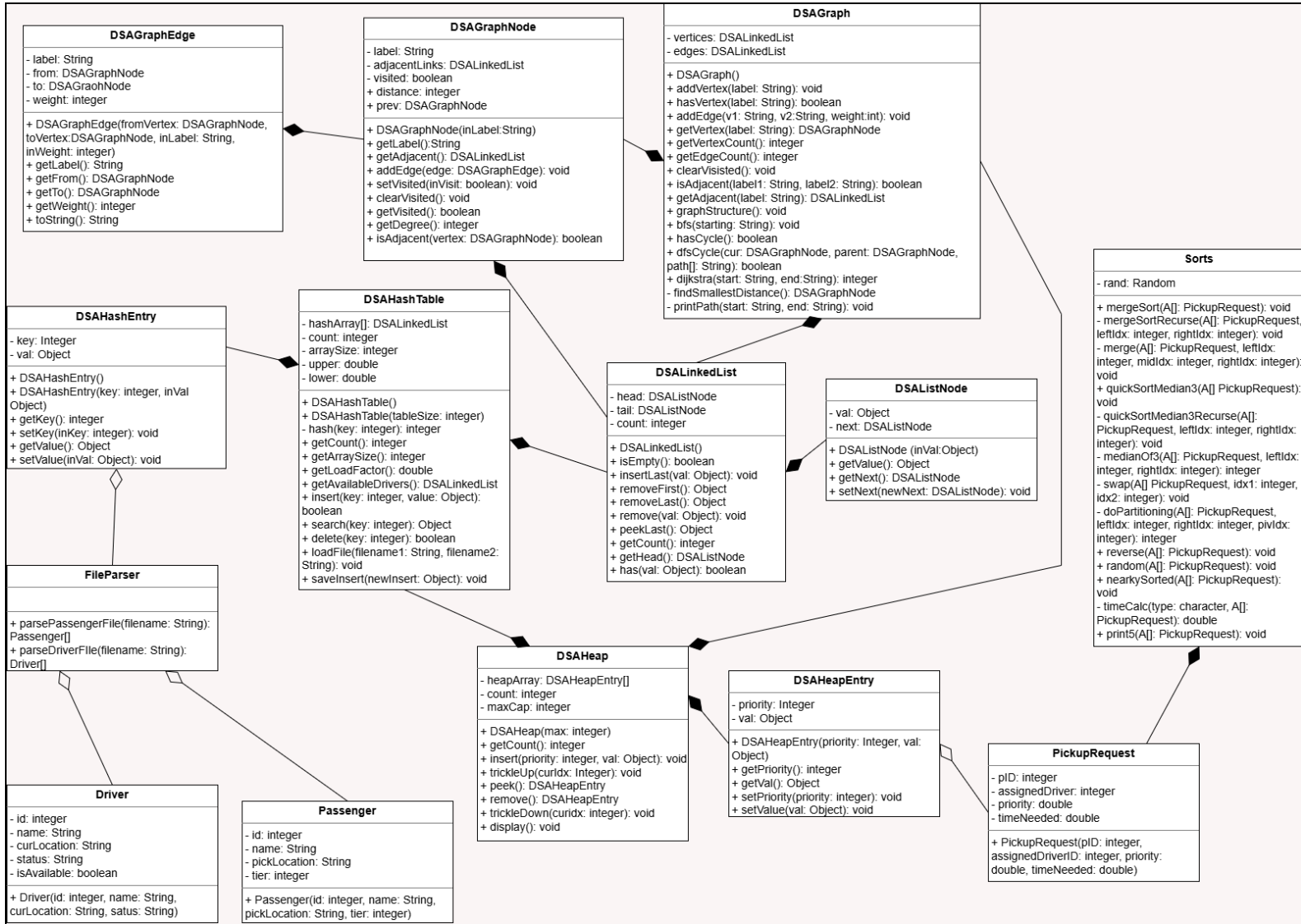
Module Integration

Graph Representation of network:



UML data-flow diagram:

COMP1002 Data Structures and Algorithms Assignment Technical Report



Algorithm complexity:

COMP1002 Data Structures and Algorithms Assignment Technical Report

Module	Algorithm	Time	Space
Graph	BFS	$O(V+E)$	$O(V)$
Graph	DFS	$O(V+E)$	$O(V)$
Graph	Dijkstra	$O(V^2)$	$O(V)$
Hash Table	Chaining	$O(1)$ for average case	$O(1)$
Heap	Insertion/Extraction	$O(\log N)$	$O(N)$
Sorting	Merge Sort	$O(N \log N)$	$O(N)$
	Quick Sort	$O(N \log N)$ for average	$O(\log N)$

Sample output

Module 1 (Graph)

```
===Graph Structure===
CBD: Airport(20), Uni(13),
Airport: CBD(20), Uni(35), SuburbSouth(17),
Uni: CBD(13), Airport(35), SuburbNorth(10), Cafe(10),
SuburbSouth: Airport(17), Mall(13),
SuburbNorth: Uni(10), Cafe(25), Hospital(23), Mall(12),
Mall: SuburbSouth(13), SuburbNorth(12),
Cafe: Uni(10), SuburbNorth(25), Hospital(30),
Hospital: Cafe(30), SuburbNorth(23),
Park:

===BFS===
CBD, Airport(Level 0)
CBD, Uni(Level 0)
Airport, SuburbSouth(Level 1)
Uni, SuburbNorth(Level 1)
Uni, Cafe(Level 1)
SuburbSouth, Mall(Level 2)
SuburbNorth, Hospital(Level 2)

===DFS and Cycle Detection===
Cycle Detected: CBD -> Airport -> Uni -> CBD

===Shortest Distance===
Start: CBD
End: Hospital
Path: CBD -> Uni -> SuburbNorth -> Hospital
Shortest distance to destination: 46 min
```

Module 2 (HashTable):

```
Main Menu
>1 Load passengers and drivers from .csv
>2 Insert Passenger or Driver
>3 Search by ID
>4 Delete ID
>5 Exit
Choice:
```

Load from file (Option 1):

COMP1002 Data Structures and Algorithms Assignment Technical Report

First 10 and Load factor:

```
Inserting passengers into hash table
Added key 160
-----
Added key 197
-----
Added key 110
-----
Added key 198
-----
Added key 170
-----
Added key 192
-----
Added key 185
-----
Added key 179
-----
Added key 164
-----
Added key 150
-----
=====
==LoadFactor Info==
Total items in hashArray: 10
Load: 0.18867924528301888
=====
```

Next 10 inserts with collision handling:

```
Added key 120
-----
Added key 156
-----
Added key 105
-----
Added key 180
-----
Added key 153
-----
Added key 190
-----
Index 21 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 127
-----
Added key 114
-----
Added key 106
-----
Added key 142
-----
=====
==LoadFactor Info==
Total items in hashArray: 20
Load: 0.37735849056603776
=====
```

Next 10 inserts from driver.csv

```
Index 20 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 497
-----
Index 47 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 471
-----
Index 14 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 438
-----
Index 5 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 429
-----
Added key 417
-----
Added key 472
-----
Added key 437
-----
Index 4 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 481
-----
Index 11 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 488
-----
```

Next 10 insertions:

```
==LoadFactor Info==
Total items in hashArray: 30
Load: 0.5660377358490566
=====
Added key 465
-----
Added key 467
-----
Added key 499
-----
Index 11 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 435
-----
Added key 489
-----
Added key 401
-----
Index 22 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 446
-----
Added key 433
-----
Index 5 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 482
-----
Index 1 in use (Colliion Detected)
Added key in the Linked List of the index
Added key 425
-----
```

```
==LoadFactor Info==  
Total items in hashArray: 40  
Load: 0.7547169811320755  
Load Factor has exceed 0.7
```

Choice 2 (Manual insert also saves in the file)

```
Do you want to insert 1) passenger or 2) driver?  
Choice: 1  
Enter ID(100-200): 123  
Enter name: Bhavisha  
Enter pick up location: Uni  
Enter tier (1-5): 4  
Added key 123
```

Choice 2 (Manual insert Fail (ID already exist))

```
Do you want to insert 1) passenger or 2) driver?  
Choice: 1  
Enter ID(100-200): 160  
Enter name: Lala  
Enter pick up location: Uni  
Enter tier (1-5): 3  
Same ID found. Not allowed
```

New insert saved in passenger.csv

```
21 123,Bhavisha,Uni,4
```

Choice 3 (Search of ID: Pass)

```
Enter ID to search: 160  
Passenger ID: 160  
Passenger Name: Chilton  
Passenger PickUp Location: Airport  
Passenger Tier: 3  
RunTime: 2554ms
```

Choice 3 (Search of ID: Fail)

```
Choice: 3  
Enter ID to search: 145  
This ID is not in the records  
RunTime: 145ms
```

Choice 4(Delete ID: Pass)

```
Choice: 4  
Enter ID to be deleted: 160  
The Data has been deleted  
RunTime: 115ms
```

Choice 4(Delete ID: Fail, doesn't exist)

```
Choice: 4  
Enter ID to be deleted: 160  
The key was not found  
RunTime: 312ms
```

When tried to search for deleted ID

```
Choice: 3  
Enter ID to search: 160  
This ID is not in the records  
RunTime: 580ms
```

Module 3 (Heap)

First the graph is made. Then all the records in the csv files are loaded and inserted in the hash table.

Then added in the heap array (10 similar inserts are done)

```

Passenger: Agnola (Tier: 3)
Driver: Ximenez
Time to reach: 0
Priority: 2003 ((6.0 - 3) + (1000.0/0.5))

Heap after insert
Heap Display
Total items: 1
Tree Structure (level by level):
Level 1: 2003,
HeapArray representation: [2003, ]

The request was added in the heapArray
    
```

There are 10 requests in heapArray but only 9 available drivers as per driver.csv. Hence it says:

```

No Driver available
    
```

Last heap insert:

```

Passenger: Paloma (Tier: 5)
Driver: Cathrine
Time to reach: 17
Priority: 59 ((6.0 - 5) + (1000.0/17.0))

Heap after insert
Heap Display
Total items: 9
Tree Structure (level by level):
Level 1: 2005,
Level 2: 2004,2003,
Level 3: 102,2003,86,38,
Level 4: 35,59,
HeapArray representation: [2005, 2004, 2003, 102, 2003, 86, 38, 35, 59, ]
    
```

This is the extract by highest priority (for testing only 5 extracts are done)

```

Dispatch 1
Priority: 2005
Passenger ID: 179
Driver ID: 401
Travel Time: 0.0

Heap Display
Total items: 8
Tree Structure (level by level):
Level 1: 2004,
Level 2: 2003,2003,
Level 3: 102,59,86,38,
Level 4: 35,
HeapArray representation: [2004, 2003, 2003, 102, 59, 86, 38, 35, ]
    
```

After 5 extracts:

```
Dispatch 5
Priority: 102
Passenger ID: 156
Driver ID: 481
Travel Time: 10.0

Heap Display
Total items: 4
Tree Structure (level by level):
Level 1: 86,
Level 2: 59,38,
Level 3: 35,
HeapArray representation: [86, 59, 38, 35, ]
```

This is the peek hughes request in the heap Array:

```
===Peek the current highest priority===
Priority: 86
PassengerID: 105
Driver ID: 437
Time Needed: 12.0
```

Module 4 (Sorting)

Testing is done for data size 100, 500 and 1000. Data generated is random and passenger ID, driver ID and priority are similar and generated for testing.

This is the sorted data for 100 records only:

```
===Sorted Data for 100 records===
Merge Sort:
-0 PassengerID: 198 | Time Needed: 2.59 min
-1 PassengerID: 171 | Time Needed: 2.61 min
-2 PassengerID: 124 | Time Needed: 2.85 min
-3 PassengerID: 172 | Time Needed: 3.15 min
-4 PassengerID: 183 | Time Needed: 5.55 min

Quick Sort (Median 3):
-0 PassengerID: 198 | Time Needed: 2.59 min
-1 PassengerID: 171 | Time Needed: 2.61 min
-2 PassengerID: 124 | Time Needed: 2.85 min
-3 PassengerID: 172 | Time Needed: 3.15 min
-4 PassengerID: 183 | Time Needed: 5.55 min
```

This is the summary table:

Test Size	Order	MergeRunTime	QuickSortRunTime
100	reversed	58.9110	33.7920
100	random	320.5640	38.8240
100	nearlySorted	8.2300	25.5710
500	reversed	94.6570	191.1770
500	random	71.7140	61.8180
500	nearlySorted	60.5180	32.3710
1000	reversed	192.6080	105.7590
1000	random	215.4640	138.2760
1000	nearlySorted	135.8230	70.8510

```

===Performance Analysis===
-> Quick Sort performed sorting faster in most of the cases, especially for the
random case
-> Merge Sort performed better for nearly sorted data for all sizes

```

Reflections:

For the implementation of the Dijkstra algorithm I had to do extra research as I didn't know how to implement it and how it works. Throughout the research this algorithm is simple to implement. Most of what I have done in this assignment has been done during the practical, so it was easier to refer. Although, some changes were required to operate the program properly.

Limitations & Assumption

Limitations:

1. Hash table is set to a fixed size of 53, resizing is not implemented here as we have 40 records.
2. Passenger and Driver data has a fixed order.
3. Passengers are given IDs from 100 - 200. Drivers are given IDs from 400 - 500. Hence the IDs and number of passengers are fixed to be within that range.

Assumptions:

1. All edges have positive weights
2. Passengers are given IDs from 100 - 200. Drivers are given IDs from 400 - 500.
3. When the driver is on the same node the minT is set to 0.5 for priority calculation purposes.